

Міністерство освіти і науки України
Національний університет "Львівська політехніка"



Звіт до лабораторної роботи №7
З дисципліни "Web-технології та Web-дизайн"
на тему: "React.js: Catalog page"

Виконала:

Ст.гр. ІР-11

Луцан І.З.

Прийняв:

доцент, к.т.н.

Іванюк О.О.

Львів 2025

Завдання:

7. React.js: Catalog page

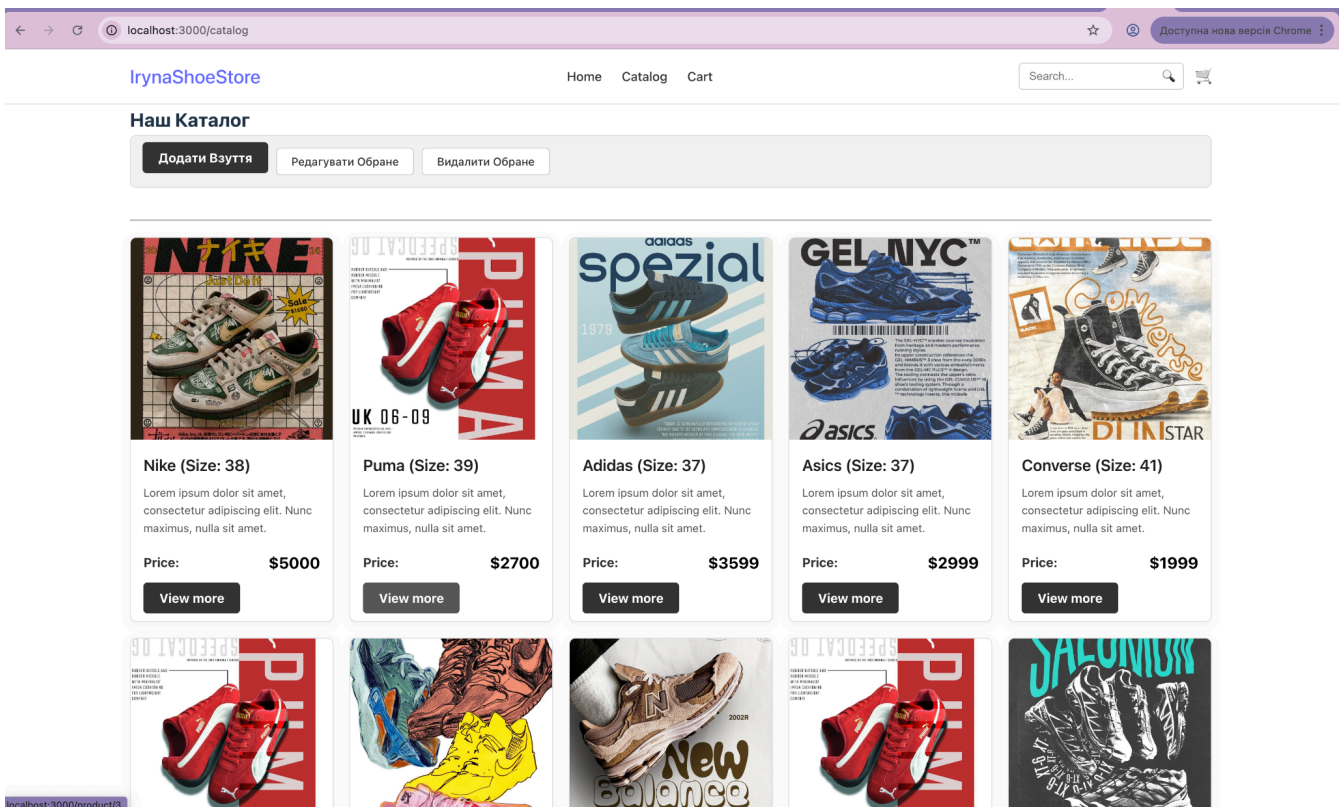
Description: Continue work on your React App by adding a page with Items list (see the link to wireframe of **Catalog page** above).

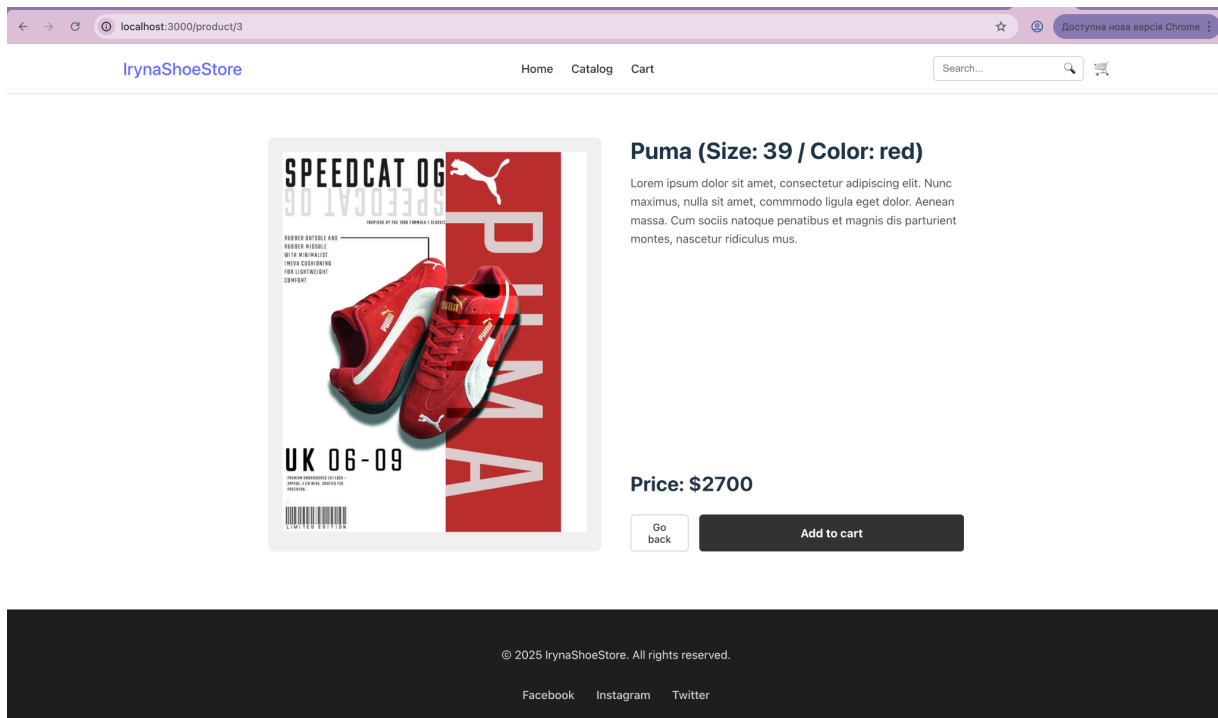
Variants - (products that you are 'selling') the same as for previous works. (see the description to 3rd work)

Requirements:

- All of the requirements for previous React.js works should be kept.
- **Code style:**
 - Use **array.map()** method for rendering your items list
 - Routing (switching between pages) should work now. Use **react-router-dom** library: <https://reactrouter.com/web/guides/quick-start>
 - All UI elements (buttons / select) should have corresponding React components (PrimaryButton.jsx / Select.jsx etc.)
- **Functionality** (filter / search / view more) is still **not required** (you have to complete it on next works)

Сайт:

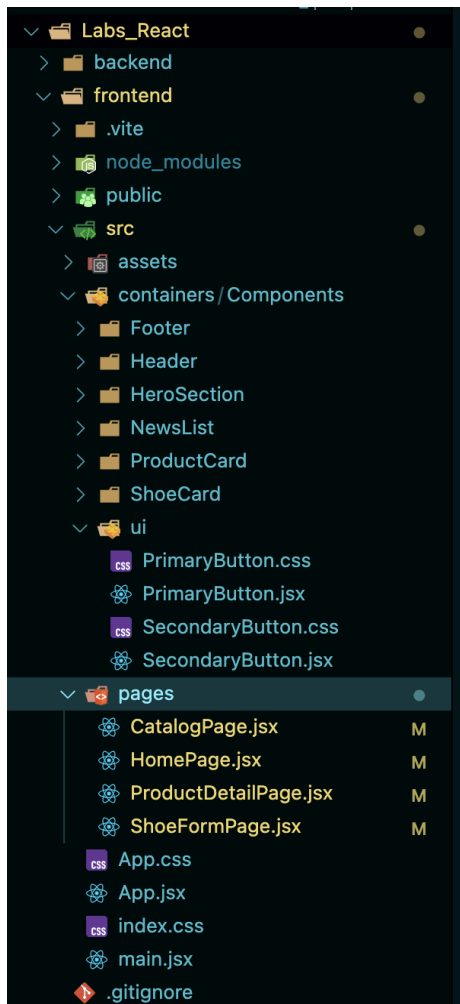




Реалізація бекенду:

файл [app.py](#) який було представлено у звіті з 6 лабораторної роботи

Структура фронтенду:



Файл CatalogPage.jsx:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import ShoeCard from '../containers/Components/ShoeCard/ShoeCard';
import SecondaryButton from '../containers/Components/ui/SecondaryButton';
import PrimaryButton from '../containers/Components/ui/PrimaryButton';
import { Link } from 'react-router-dom';

// Стилі для СІТКИ (Grid)
const gridStyles = {
  display: 'grid',
  gridTemplateColumns: 'repeat(auto-fill, minmax(250px, 1fr))',
  gap: '20px',
  padding: '20px 0'
};

// Стилі для панелі фільтрів
const filterBarStyles = {
  display: 'flex',
  flexWrap: 'wrap',
  gap: '15px',
  alignItems: 'center',
  padding: '10px 0'
};

// Стилі для адмін-панелі
const adminPanelStyles = {
  padding: '15px',
  backgroundColor: '#f4f4f4',
  border: '1px solid #ddd',
  borderRadius: '8px',
  display: 'flex',
  gap: '10px',
  marginBottom: '20px'
};

function CatalogPage() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [selectedId, setSelectedId] = useState(null);
  const [isEditDisabled, setIsEditDisabled] = useState(true);
  const [isDeleteDisabled, setIsDeleteDisabled] = useState(true);

  useEffect(() => {
    setIsEditDisabled(selectedId === null);
    setIsDeleteDisabled(selectedId === null);
  }, [selectedId]);

  // виносим 'fetchProducts' з 'useEffect',
  // щоб мати можливість викликати її повторно
```

```

const fetchProducts = async () => {
  try {
    setLoading(true);
    setError(null);
    const response = await axios.get('/api/shoes');
    setProducts(response.data);
  } catch (err) {
    setError(err.message);
    console.error("Помилка завантаження взуття:", err);
  } finally {
    setLoading(false);
  }
};

// викликає 'fetchProducts' при першому завантаженні
useEffect(() => {
  fetchProducts();
}, []); // [] - виконати 1 раз при завантаженні

// видалення
const handleDelete = async () => {
  // Перевірка, чи щось обрано
  if (!selectedId) {
    alert("Будь ласка, оберіть товар для видалення.");
    return;
  }

  // 4. Запитуємо підтвердження
  if (window.confirm('Ви впевнені, що хочете видалити цей товар?')) {
    try {
      // робим запит до API на видалення
      await axios.delete(`/api/shoes/${selectedId}`);

      setSelectedId(null);

      alert('Товар успішно видалено!');
      fetchProducts(); // Викликаємо завантаження даних
    } catch (err) {
      console.error("Помилка видалення:", err);
      alert("Не вдалося видалити товар: " + err.message);
    }
  }
};

if (loading) return <p>Завантаження товарів...</p>;
if (error) return <p>Сталася помилка: {error}</p>;

return (

```

```

<div>
  <h2>Наш Каталог</h2>

  <div style={adminPanelStyles}>
    <Link to="/catalog/new" style={{ textDecoration: 'none' }}>
      <PrimaryButton as="span"> {/* 'as="span"' змусить кнопку вести себе як блок */}
        Додати Взуття
      </PrimaryButton>
    </Link>

    <SecondaryButton disabled={isEditDisabled}>
      Редагувати Обране
    </SecondaryButton>
    <SecondaryButton
      disabled={isDeleteDisabled}
      style={!isDeleteDisabled ? {backgroundColor: '#ffdddd', borderColor: '#ff5555',
color: '#d00'} : {}}
      onClick={handleDelete}
    >
      Видалити Обране
    </SecondaryButton>
  </div>

  <div style={filterBarStyles}>

</div>
<hr />

<div style={gridStyles}>
  {products.map(product => (
    <ShoeCard
      key={product.id}
      product={product}
      onSelect={() => {
        // Додамо логіку "зняття виділення", якщо клікнути на ту саму картку
        setSelectedId(prevId => prevId === product.id ? null : product.id);
      }}
      isSelected={product.id === selectedId}
    />
  ))}
</div>
</div>
);
}

export default CatalogPage;

```

Файл ShoeForm.jsx:

```
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import PrimaryButton from '../containers/Components/ui/PrimaryButton';
import SecondaryButton from '../containers/Components/ui/SecondaryButton';

const formContainerStyles = {
  maxWidth: '600px',
  margin: '20px auto',
  padding: '20px',
  border: '1px solid #ddd',
  borderRadius: '8px',
  backgroundColor: '#f9f9f9',
};

const formGroupStyles = {
  marginBottom: '15px',
};

const labelStyles = {
  display: 'block',
  marginBottom: '5px',
  fontWeight: 'bold',
};

const inputStyles = {
  width: '100%',
  padding: '8px',
  border: '1px solid #ccc',
  borderRadius: '4px',
};

const buttonGroupStyles = {
  display: 'flex',
  gap: '10px',
};

function ShoeFormPage() {
  // 'навігатор'
  const navigate = useNavigate();

  // стан для кожного поля форми
  const [producer, setProducer] = useState('');
  const [price, setPrice] = useState('');
  const [size, setSize] = useState('');
  const [color, setColor] = useState('');
  const [error, setError] = useState(null); // Для показу помилок

  // функція, викликається при відправці форми
```

```

const handleSubmit = async (e) => {
  e.preventDefault();

  if (!producer || !price || !size || !color) {
    setError('Будь ласка, заповніть усі поля');
    return;
  }

  const newShoe = {
    producer: producer.trim(),
    price: parseFloat(price),
    size: parseInt(size),
    color: color.trim(),
  };

  try {
    await axios.post('/api/shoes', newShoe);
    alert('Товар успішно додано!');
    navigate('/catalog');

  } catch (err) {
    setError(err.message);
    console.error("Помилка додавання товару:", err);
  }
};

return (
  <div style={formContainerStyles}>
    <h2>Додати нове взуття</h2>

    {/* 'onSubmit' прив'язує функцію до форми */}
    <form onSubmit={handleSubmit}>

      {/* Поле Producer */}
      <div style={formGroupStyles}>
        <label style={labelStyles} htmlFor="producer">Виробник:</label>
        <input
          style={inputStyles}
          type="text"
          id="producer"
          value={producer}
          onChange={e => setProducer(e.target.value)}
        />
      </div>

      {/* Поле Price */}
      <div style={formGroupStyles}>
        <label style={labelStyles} htmlFor="price">Ціна:</label>
        <input
          style={inputStyles}
          type="number"

```



```

        id="price"
        value={price}
        onChange={ (e) => setPrice(e.target.value) }
      />
    </div>

    { /* Поле Size */ }
    <div style={formGroupStyles}>
      <label style={labelStyles} htmlFor="size">Розмір:</label>
      <input
        style={inputStyles}
        type="number"
        id="size"
        value={size}
        onChange={ (e) => setSize(e.target.value) }
      />
    </div>

    <div style={formGroupStyles}>
      <label style={labelStyles} htmlFor="color">Колір:</label>
      <input
        style={inputStyles}
        type="text"
        id="color"
        value={color}
        onChange={ (e) => setColor(e.target.value) }
      />
    </div>

    { /* показ помилок валідації */ }
    {error && <p style={{ color: 'red' }}>{error}</p>}

    <div style={buttonGroupStyles}>
      <PrimaryButton type="submit">Зберегти</PrimaryButton>
      <SecondaryButton type="button" onClick={ () => navigate('/catalog') }>
        Скасувати
      </SecondaryButton>
    </div>

  </form>
</div>
);
}

export default ShoeFormPage;

```

Посилання на лабораторну роботу на гітхабі:

https://github.com/IrynaLutsan/Web_Programming_Labs/tree/main/Labs_React

Висновок:

У цій лабораторній роботі було продовжено розробку React-застосунку шляхом створення сторінки каталогу з переліком товарів, реалізованим через метод `array.map()`. Було налаштовано роутинг між сторінками за допомогою бібліотеки `react-router-dom`, що забезпечило коректну навігацію всередині застосунку. Усі елементи інтерфейсу були винесені в окремі React-компоненти, що зробило структуру проєкту зрозумілішою й підтримуваною. Функціональні можливості, такі як фільтрація, пошук чи перегляд деталізації, поки не вимагалися, тому основний акцент був зроблений на правильній організації коду та відтворенні базового відображення каталогу.